Solution: Equal Subset Sum Partition

Problem Statement

Given a set of positive numbers, find if we can partition it into two subsets such that the sum of elements in both the subsets is equal.

Example 1:

```
Input: {1, 2, 3, 4}
Output: True
Explanation: The given set can be partitioned into two subsets with
equal sum: {1, 4} & {2, 3}
```

Example 2:

```
Input: {1, 1, 3, 4, 7}
Output: True
Explanation: The given set can be partitioned into two subsets with
equal sum: {1, 3, 4} & {1, 7}
```

Example 3:

```
Input: {2, 3, 4, 6}
Output: False
Explanation: The given set cannot be partitioned into two subsets with
equal sum.
```

Basic Solution

This problem follows the **0/1 Knapsack pattern.** A basic brute-force solution could be to try all combinations of partitioning the given numbers into two sets to see if any pair of sets has an equal sum.

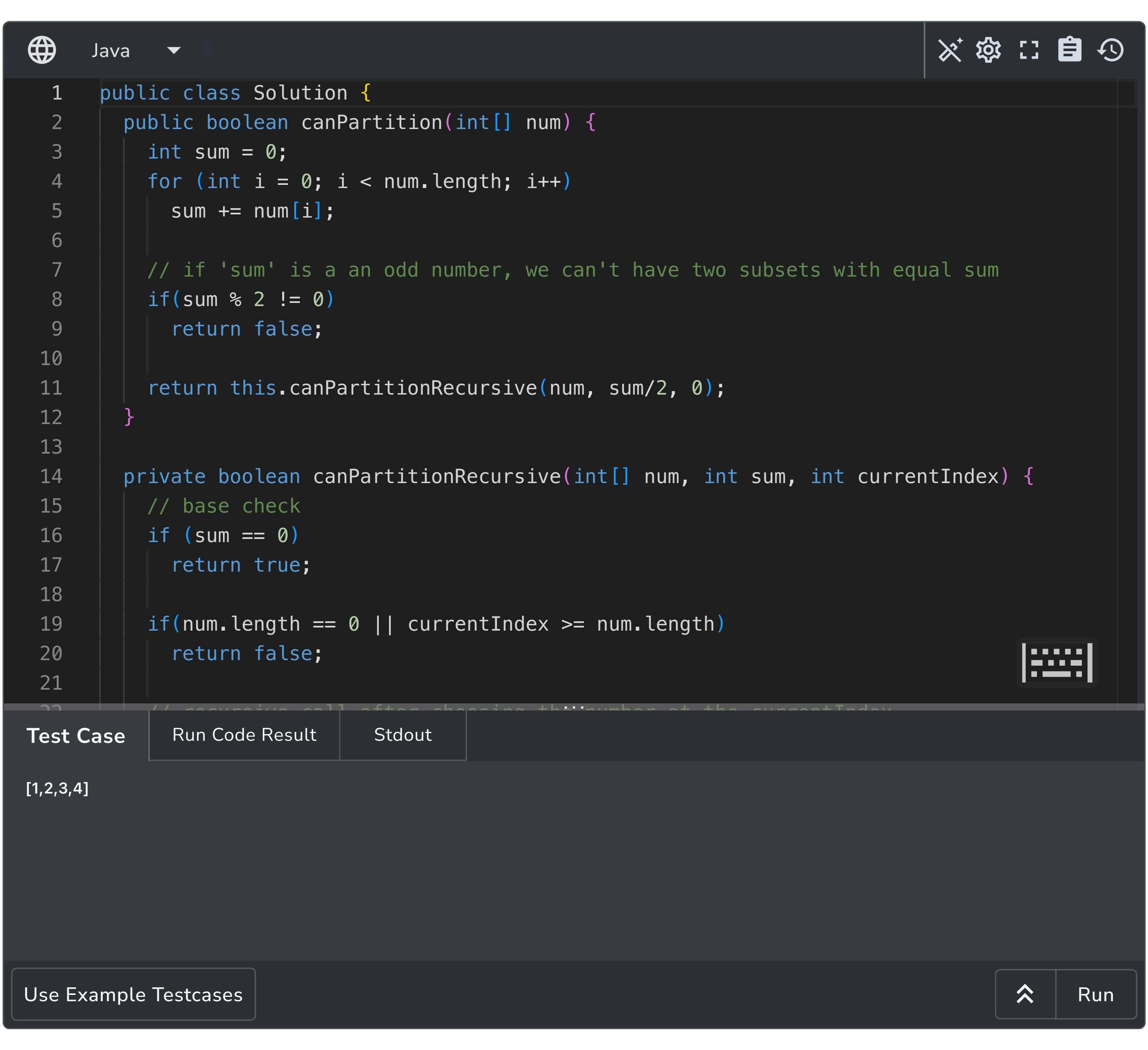
Assume if `S` represents the total sum of all the given numbers, then the two equal subsets must have a sum equal to `S/2`. This essentially transforms our problem to: "Find a subset of the given numbers that has a total sum of `S/2`".

So our brute-force algorithm will look like:

```
for each number 'i'
  create a new set which INCLUDES number 'i' if it does not exceed 'S/2', and
recursively
      process the remaining numbers
  create a new set WITHOUT number 'i', and recursively process the remaining items
return true if any of the above sets has a sum equal to 'S/2', otherwise return false
```

Code

Here is the code for the brute-force solution:



```java
public class Solution {
  public boolean canPartition(int[] num) {
    int sum = 0;
    for (int i = 0; i < num.length; i++)
      sum += num[i];

    // if 'sum' is a an odd number, we can't have two subsets with equal sum
    if(sum % 2 != 0)
      return false;

    return this.canPartitionRecursive(num, sum/2, 0);
  }

  private boolean canPartitionRecursive(int[] num, int sum, int currentIndex) {
    // base check
    if (sum == 0)
      return true;

    if(num.length == 0 || currentIndex >= num.length)
      return false;

```

[1,2,3,4]

The time complexity of the above algorithm is exponential $O(2^n)$, where 'n' represents the total number. The space complexity is $O(n)$, this memory which will be used to store the recursion stack.

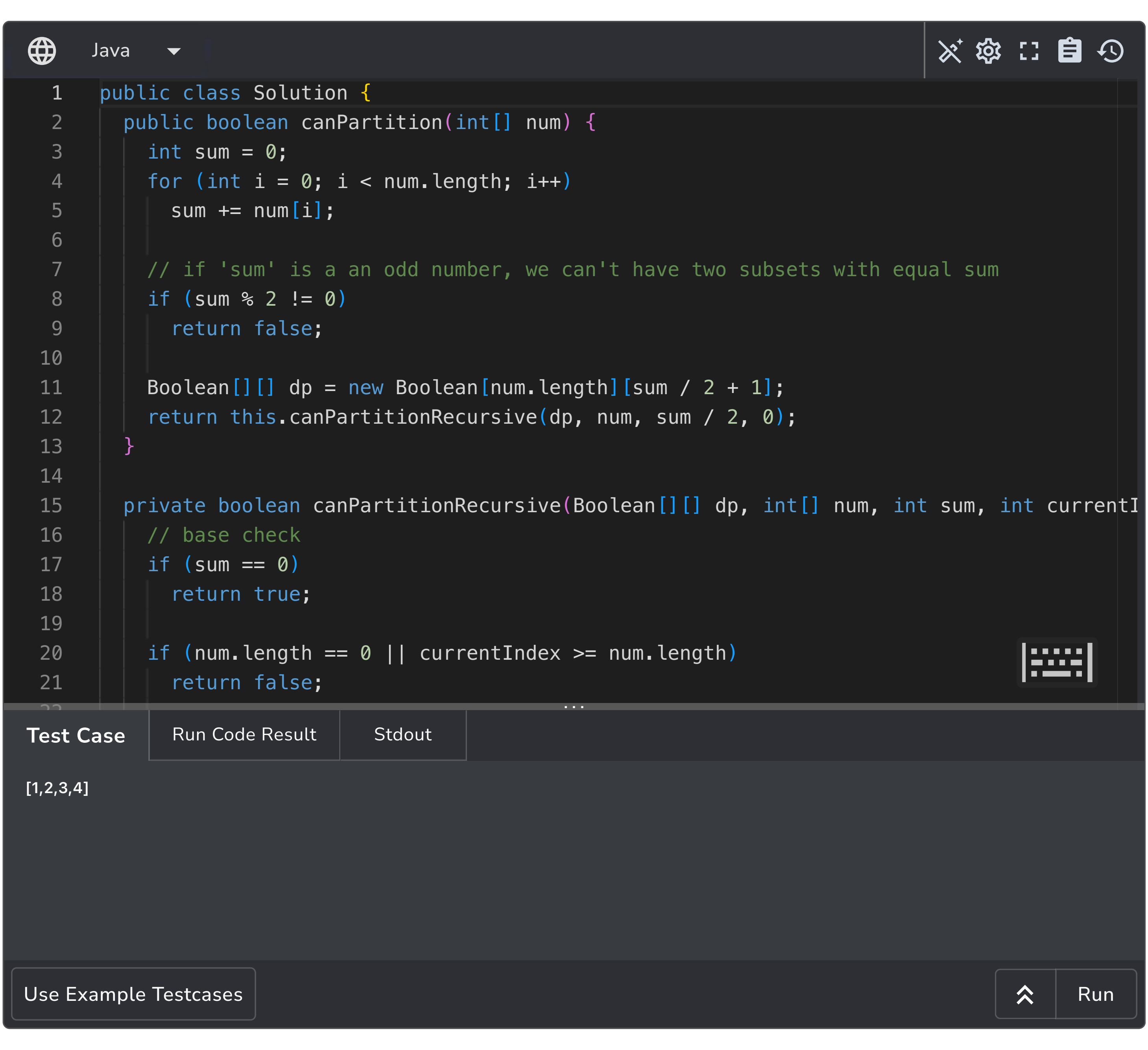
Top-down Dynamic Programming with Memoization

We can use memoization to overcome the overlapping sub-problems. As stated in previous lessons, memoization is when we store the results of all the previously solved sub-problems return the results from memory if we encounter a problem that has already been solved.

Since we need to store the results for every subset and for every possible sum, therefore we will be using a two-dimensional array to store the results of the solved sub-problems. The first dimension of the array will represent different subsets and the second dimension will represent different 'sums' that we can calculate from each subset. These two dimensions of the array can also be inferred from the two changing values (sum and currentIndex) in our recursive function `canPartitionRecursive()`.

Code

Here is the code for Top-down DP with memoization:



```java
public class Solution {
  public boolean canPartition(int[] num) {
    int sum = 0;
    for (int i = 0; i < num.length; i++)
      sum += num[i];

    // if 'sum' is a an odd number, we can't have two subsets with equal sum
    if (sum % 2 != 0)
      return false;

    Boolean[][] dp = new Boolean[num.length][sum / 2 + 1];
    return this.canPartitionRecursive(dp, num, sum / 2, 0);
  }

  private boolean canPartitionRecursive(Boolean[][] dp, int[] num, int sum, int currentI
    // base check
    if (sum == 0)
      return true;

    if (num.length == 0 || currentIndex >= num.length)
      return false;
```

[1,2,3,4]

The above algorithm has time and space complexity of $O(N * S)$, where 'N' represents total numbers and 'S' is the total sum of all the numbers.

Bottom-up Dynamic Programming

Let's try to populate our `dp[][]` array from the above solution, working in a bottom-up fashion. Essentially, we want to find if we can make all possible sums with every subset. **This means,**

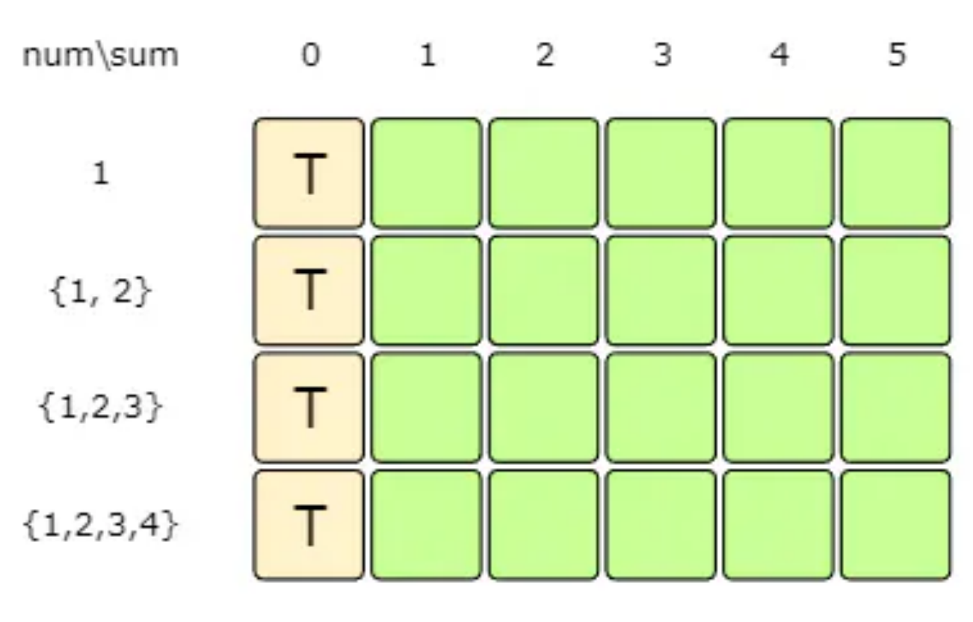
`dp[i][s]` **will be 'true' if we can make sum 's' from the first 'i' numbers.**

So, for each number at index 'i' (0 <= i < num.length) and sum 's' (0 <= s <= S/2), we have two options:

1. Exclude the number. In this case, we will see if we can get 's' from the subset excluding this number: `dp[i-1][s]`
2. Include the number if its value is not more than 's'. In this case, we will see if we can find a subset to get the remaining sum: `dp[i-1][s-num[i]]`

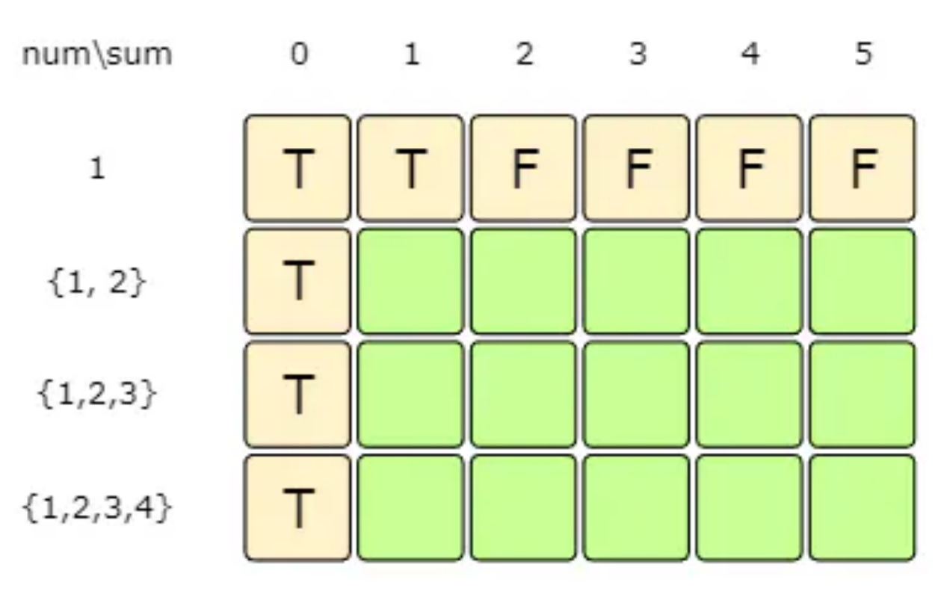
If either of the two above scenarios is true, we can find a subset of numbers with a sum equal to 's'.

Let's start with our base case of zero capacity:



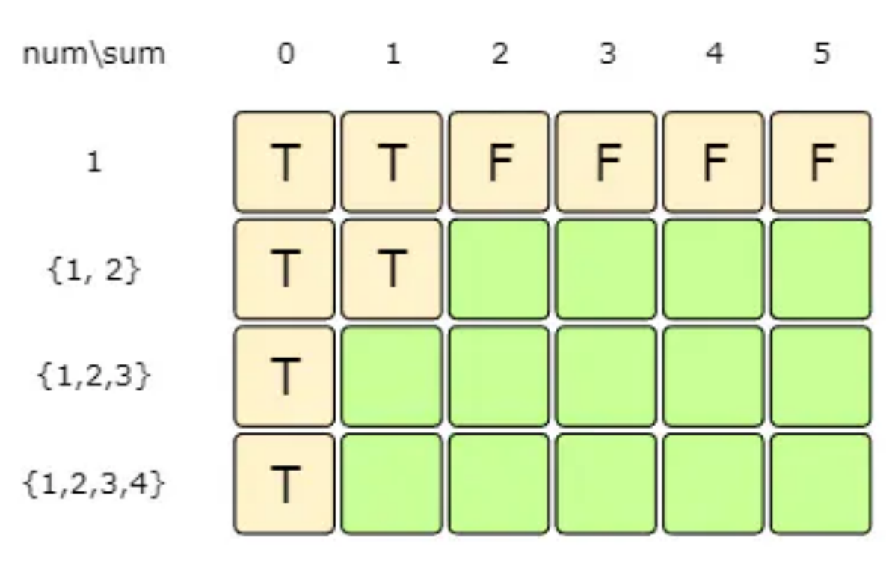
num\sum	0	1	2	3	4	5
1	T					
{1, 2}	T					
{1,2,3}	T					
{1,2,3,4}	T					

'0' sum can always be found through an empty set



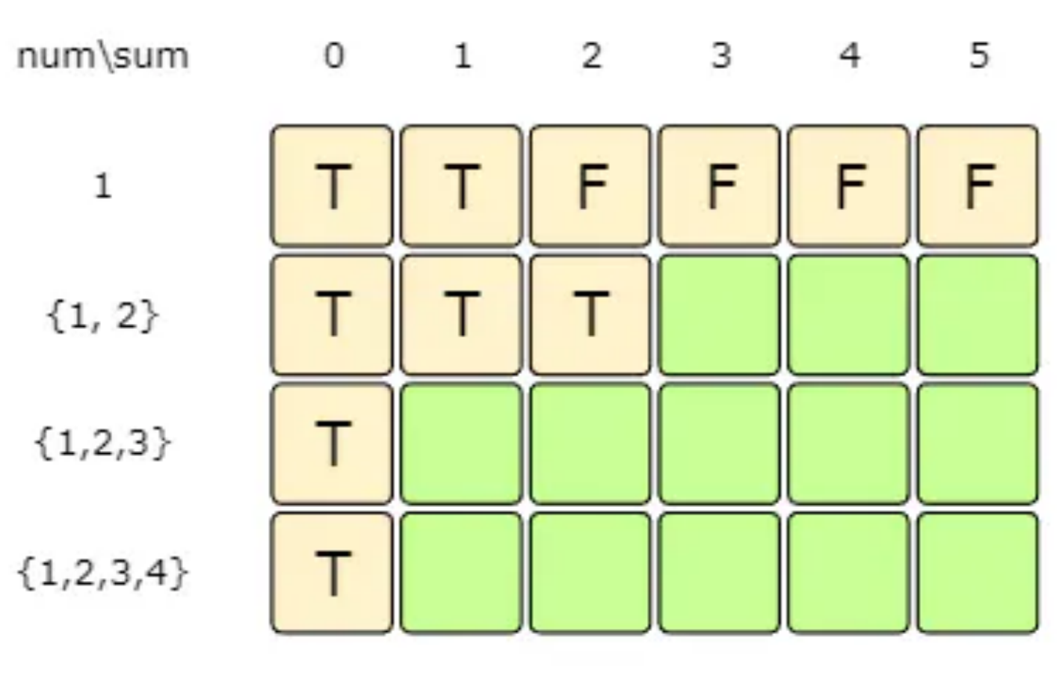
num\sum	0	1	2	3	4	5
1	T	T	F	F	F	F
{1, 2}	T					
{1,2,3}	T					
{1,2,3,4}	T					

With only one number, we can form a subset only when the required sum is equal to its value



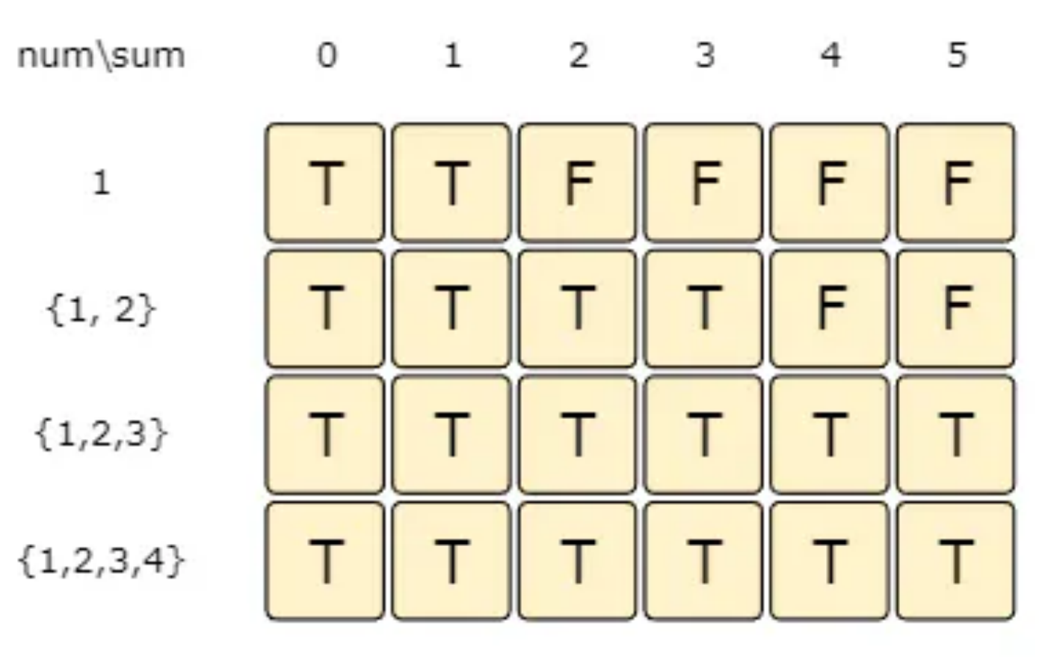
num\sum	0	1	2	3	4	5
1	T	T	F	F	F	F
{1, 2}	T	T				
{1,2,3}	T					
{1,2,3,4}	T					

sum: 1, index:1=> (dp[index-1][sum] , as the 'sum' is less than the number at index '1'



num\sum	0	1	2	3	4	5
1	T	T	F	F	F	F
{1, 2}	T	T	T			
{1,2,3}	T					
{1,2,3,4}	T					

sum: 2, index:1=> (dp[index-1][sum] || dp[index-1][sum-2])



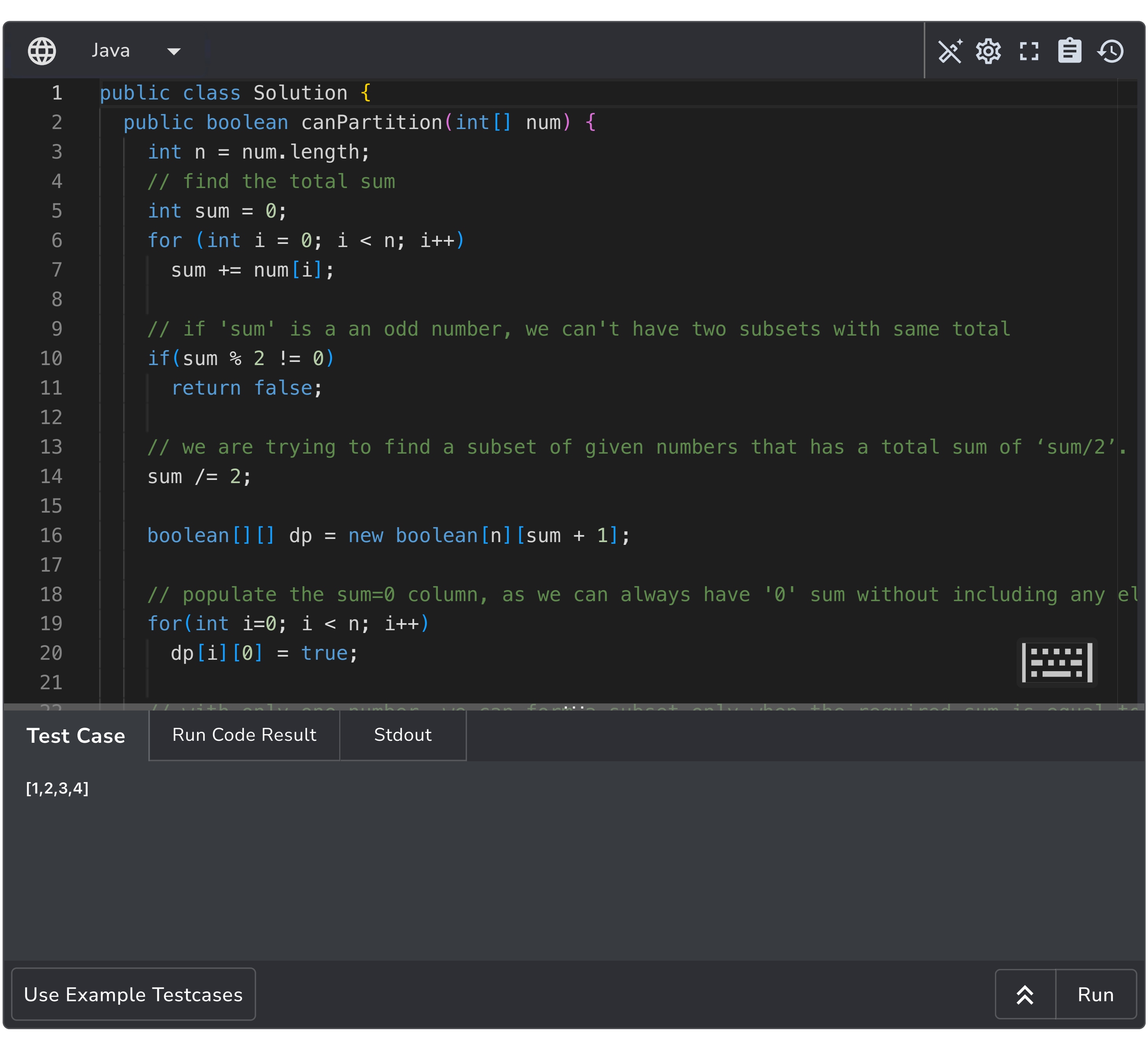
num\sum	0	1	2	3	4	5
1	T	T	F	F	F	F
{1, 2}	T	T	T	T	F	F
{1,2,3}	T	T	T	T	T	T
{1,2,3,4}	T	T	T	T	T	T

sum: 1-5, index:3=> (dp[index-1][sum] || dp[index-1][sum-4])

From the above visualization, we can clearly see that it is possible to partition the given set into two subsets with equal sums, as shown by bottom-right cell: `dp[3][5] => T`

Code

Here is the code for our bottom-up dynamic programming approach:



```java
public class Solution {
  public boolean canPartition(int[] num) {
    int n = num.length;
    // find the total sum
    int sum = 0;
    for (int i = 0; i < n; i++)
      sum += num[i];

    // if 'sum' is a an odd number, we can't have two subsets with same total
    if(sum % 2 != 0)
      return false;

    // we are trying to find a subset of given numbers that has a total sum of ‘sum/2’.
    sum /= 2;

    boolean[][] dp = new boolean[n][sum + 1];

    // populate the sum=0 column, as we can always have '0' sum without including any el
    for(int i=0; i < n; i++)
      dp[i][0] = true;

```

[1,2,3,4]

The above solution has time and space complexity of $O(N * S)$, where 'N' represents total numbers and 'S' is the total sum of all the numbers.

← Previous
Equal Subset Sum Partition
Next →
Subset Sum
☐ Mark as Completed